# Participant Prerequisites

## AI-Augmented Product Engineering Program

To ensure that the three-day hands-on program runs smoothly, participants should complete the following prerequisites before the training begins.

## 1. Participant Laptop

Each participant must have an individual laptop with:

- Windows 10/11 or macOS 13 or later
- Minimum 8 GB RAM; 16 GB recommended.
- Minimum 10 GB free disk space
- Stable internet connectivity
- Permission to install and run approved development tools
- Permission to create Python virtual environments
- Permission to run applications on local `localhost` ports

Administrative access may be required during initial software installation.

## 2. Personal GitHub Account

Each participant must have:

- An active personal GitHub account
- Ability to sign in to GitHub.
- Ability to create a private GitHub repository.
- Ability to clone, commit, push and create pull requests.
- GitHub credentials or a supported authentication method configured on the laptop.

Participants will create a private repository from the training template and maintain their individual lab work in that repository.

No client source code, customer data or confidential documentation will be used or uploaded to personal GitHub accounts. All supplied training material will be fictional and synthetic.

## 3. Required Software

The following software must be installed before the program:

| Software | Requirement |
|---|---|
| Python | Python 3.11.x, 64-bit |
| Visual Studio Code | Latest stable version |
| Git | Latest stable version |

| Software | Requirement |
|---|---|
| Web browser | Latest Chrome or Microsoft Edge |
| VS Code Python extension | Microsoft Python extension |
| VS Code GitHub extension | Recommended |
| Pip | Included with Python |
| Python virtual environment support | `venv` must be available |

# 4. VS Code Requirements

Participants should be able to:

- Open a local project folder in VS Code
- Open and use the integrated terminal
- Select a Python interpreter
- Create and activate a Python virtual environment
- Install Python packages
- Run Python applications
- Execute Pytest tests
- Use Git source-control functions
- Clone and synchronize a GitHub repository

Recommended VS Code extensions:

- Python by Microsoft
- Pylance by Microsoft
- GitHub Pull Requests
- Jupyter, if notebooks are included
- Markdown Preview, optional

# 5. Python Environment

Participants must be able to execute the following commands:

```
python --version
pip --version
git --version
```

The Python version should return:

```
Python 3.11.x
```

Participants must also be permitted to create and activate a virtual environment:

### Windows

```
python -m venv .venv
.venv\Scripts\activate
```

### macOS

```
python3 -m venv .venv
source .venv/bin/activate
```

All required Python packages will be provided through the training repository's dependency file.

# 6. Network Access

The participant network must allow HTTPS access to:

- `github.com`
- `api.github.com`
- `raw.githubusercontent.com`
- `objects.githubusercontent.com`
- `pypi.org`
- `files.pythonhosted.org`
- The trainer-provided LLM endpoint
- Any organization-approved authentication endpoint required by the LLM service

Git operations over HTTPS must be permitted.

If the client uses a VPN, proxy, firewall or endpoint-security solution, GitHub access, Python package installation and LLM endpoint connectivity should be tested before the training.

# 7. LLM Endpoint Access

The trainer will provide:

- LLM endpoint URL
- Model/deployment name
- API version, if required
- Temporary API key or access credential
- Configuration instructions

Participants must not store the key in Python files, notebooks, screenshots or GitHub.

Credentials will be maintained locally in a `.env` file:

```
LLM_ENDPOINT=
LLM_API_KEY=
LLM_MODEL=
```

```
LLM_API_VERSION=
```

The `.env` file will be excluded from Git through `.gitignore`.

Any temporary training credentials should be restricted, monitored and revoked or rotated after the program.

# 8. Data and Security Requirements

The program will use only:

- Fictional PharmaLab application code
- Synthetic laboratory data
- Mock SAP integration
- Sanitized application and interface logs
- Training-specific SOPs and product documents
- Simulated support incidents

Participants must not introduce:

- Client proprietary source code
- Real customer information
- Patient or personal information
- Production GxP records
- Confidential product documentation
- Production credentials
- Real application logs
- Any data restricted by client policy

# 9. Pre-Training Validation

Participants will receive an environment-validation script before the program.

The script will verify:

- Python version
- Pip availability
- Git installation
- Required Python packages
- GitHub repository access
- LLM endpoint connectivity
- Required training files
- Availability of local ports
- Local application startup
- Basic test execution

Each participant should run:

```
python run_environment_check.py
```

The environment should return a successful validation result before the training begins.

# 10. Pre-Training Readiness Checklist

Each participant should confirm the following:

- Personal GitHub account is active
- A private GitHub repository can be created
- GitHub can be accessed from the training network
- Git clone, commit and push operations are working
- Python 3.11.x is installed
- VS Code is installed
- Git is installed
- Required VS Code extensions are installed
- A Python virtual environment can be created
- Python packages can be installed
- Localhost applications can be opened in a browser
- Required local ports are available
- Trainer-provided LLM endpoint is accessible
- Environment-validation script completes successfully